

CS-405: Deep Learning

Word Embeddings: From One-hot to Dense Representation

Dr. Sajjad Hussain

SEECS, NUST

April 7th, 2026

Lecture Outline

- 1 Introduction
- 2 Learning Word Embeddings
- 3 Practical Learning Approach using Negative Sampling
- 4 Word2Vec
- 5 GloVe

The Core Problem

Computers don't understand words

Computers work with **numbers**, not meaning. We need to convert text into numerical form.

Key question: How do we represent words so that:

- Similar words have similar representations
- The representation captures meaning
- Mathematical operations reveal relationships

We need a mapping: Word $\rightarrow$ Vector

Simple Approach 1: Word Indices

Assign each word a unique integer

Word	Index
king	0
queen	1
man	2
woman	3
throne	4

Example: "king sat on throne" $\rightarrow [0, 5, 6, 4]$

Problems:

- No notion of similarity (king=0, queen=1 — arbitrarily different)
- No meaning captured
- Model thinks 1 is "greater than" 0 (nonsensical)

Simple Approach 2: One-Hot Vectors

Each word becomes a binary vector

For a vocabulary of size $|V| = 5$:

king = [1, 0, 0, 0, 0]

queen = [0, 1, 0, 0, 0]

man = [0, 0, 1, 0, 0]

woman = [0, 0, 0, 1, 0]

throne = [0, 0, 0, 0, 1]

Properties:

- All vectors are orthogonal (dot product = 0)
- Each dimension corresponds to one word
- Simple and interpretable

The Problem with One-Hot Vectors

Vectors are orthogonal

$$\text{king} \cdot \text{queen} = 0$$

$$\text{man} \cdot \text{woman} = 0$$

Meaning: No similarity between any words!

Result:

- "king" and "queen" are as different as "king" and "pizza"
- Cannot learn relationships
- Vocabulary size = vector length (huge!)

With $|V| = 50,000$, each vector has 50,000 dimensions!

The Curse of Dimensionality

One-hot vectors are extremely sparse

- Vocabulary size $|V| \approx 50,000$ to $1,000,000$
- Only one "1" per vector, rest are zeros
- Most dimensions contain no information
- Models need huge amounts of data

Example with $|V| = 50,000$:

$$\text{king} = [\underbrace{1, 0, 0, \dots, 0}_{50,000 \text{ dimensions}}]$$

We need a dense, low-dimensional representation!

The Distributional Hypothesis

Firth (1957):

"You shall know a word by the company it keeps"

Meaning:

- Words that appear in similar contexts have similar meanings
- The meaning of a word is defined by its surrounding words
- We can learn representations from word co-occurrence patterns

Example:

- "king" appears near: *throne, crown, royal, queen, palace*
- "queen" appears near: *throne, crown, royal, king, palace*
- Similar contexts → similar meanings

Intuition: Word Embeddings

Dense, low-dimensional vectors

Instead of 50,000 dimensions, use $d \approx 50$ to 300 dimensions.

Key insight: Each dimension learns a **latent feature**:

Word	Gender	Royalty	Status
king	+0.8	+0.9	+0.7
queen	-0.7	+0.9	+0.7
man	+0.7	-0.8	-0.6
woman	-0.8	-0.7	-0.6

The model discovers these features automatically from text!

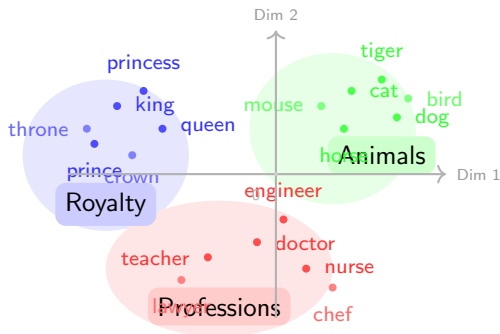
Visualizing Word Embeddings

In 2D (t-SNE projection):

- Similar words cluster together
- Relationships become geometric

Example clusters:

- **Royalty**: king, queen, prince, princess
- **Professions**: doctor, nurse, teacher, engineer
- **Animals**: cat, dog, horse, tiger



t-SNE projection of word embeddings in 2D space — similar words form clusters.

Word Embeddings Enable Vector Arithmetic

The Key Insight

If embeddings capture meaning, then **semantic relationships** should correspond to **vector operations**.

Classic analogy task:

"king is to queen as man is to _____?"

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Vector Arithmetic: Step by Step

The Analogy:

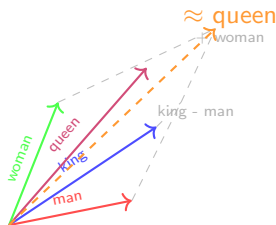
- king : queen :: man : ?
- "king is to queen as man is to
----"

Vector Operation:

$$\mathbf{v}_{king} - \mathbf{v}_{man} + \mathbf{v}_{woman} \approx \mathbf{v}_{queen}$$

Intuition:

- $\mathbf{v}_{king} - \mathbf{v}_{man}$ removes
"maleness"
- Adding $\mathbf{v}_{woman}$ adds
"femaleness"
- Result should be the female
equivalent of king



The vector difference captures the underlying semantic relationship

Concrete Numerical Example

Hypothetical 3D Embeddings

Word	Dim1 (Gender)	Dim2 (Royalty)	Dim3 (Status)
king	0.8	0.9	0.7
queen	-0.7	0.9	0.7
man	0.7	-0.8	-0.6
woman	-0.8	-0.7	-0.6

Compute: king - man + woman

$$\begin{aligned}\mathbf{v}_{king} - \mathbf{v}_{man} + \mathbf{v}_{woman} &= [0.8 - 0.7 + (-0.8), 0.9 - (-0.8) + (-0.7), 0.7 - (-0.6)] \\ &= [-0.7, 1.0, 0.7]\end{aligned}$$

$$[-0.7, 1.0, 0.7] \approx \mathbf{v}_{queen} = [-0.7, 0.9, 0.7]$$

✓ The result is very close to the actual queen embedding!

More Analogy Examples

Capital-Country:

Paris – France + Germany $\approx$ Berlin

Pluralization:

apples – apple + car $\approx$ cars

Verb Tense:

running – run + eat $\approx$ eating

Opposites:

hot – cold + good $\approx$ bad

Comparative:

better – good + bad $\approx$ worse

Professions:

actor – man + woman $\approx$ actress

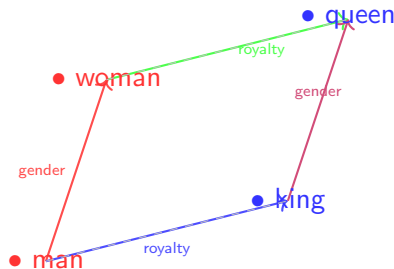
Why This Works

The embedding space learns **directions** that correspond to semantic relationships:

- Gender direction: $\mathbf{v}_{man} - \mathbf{v}_{woman}$
- Plural direction: $\mathbf{v}_{apples} - \mathbf{v}_{apple}$
- Tense direction: $\mathbf{v}_{running} - \mathbf{v}_{run}$

Visualizing the Analogy Space

Gender Analogy as Vector Offset:



Parallel vectors indicate
same relationship (gender)

Key Observation:

- $\mathbf{v}_{\text{woman}} - \mathbf{v}_{\text{man}} \approx \mathbf{v}_{\text{queen}} - \mathbf{v}_{\text{king}}$
- The **difference vectors** are nearly identical
- The embedding space is **linear**
- Analogies become simple vector addition

$$\text{king} + (\text{woman} - \text{man}) \approx \text{queen}$$

Limitations of Vector Arithmetic

Not all analogies work perfectly:

Analogy	Success Rate
king : queen	High
running : run	Medium
good : best	Low

Why some fail:

- **Multi-step relationships:** "grandfather" requires two gender steps
- **Rare words:** Insufficient training data
- **Non-linear relationships:** Some semantic changes aren't linear
- **Dataset bias:** "doctor : man :: nurse : woman" (reflects societal bias)

Takeaway

Vector arithmetic is **surprisingly powerful** but not perfect. It reveals both the strengths and biases of learned embeddings.

Sparse vs Dense: A Comparison

Property	One-Hot	Embedding
Dimension size	$ V $ (50k-1M)	d (50-300)
Sparsity	Very sparse	Dense
Similarity	None (orthogonal)	Cosine similarity
Captures meaning	No	Yes
Trainable	No	Yes
Interpretability	High (word indices)	Low (latent features)

Word embeddings trade interpretability for power and efficiency.

Summary: Why Word Embeddings?

- ① **Semantic meaning:** Similar words have similar vectors
- ② **Dense representations:** Small, efficient vectors
- ③ **Transfer learning:** Pre-trained embeddings work for many tasks
- ④ **Analogies:** Vector arithmetic reveals relationships
- ⑤ **Continuous space:** Enables neural networks to process language

The Goal

Learn a mapping $f : \text{Word} \rightarrow \mathbb{R}^d$ such that:

$$\text{similarity}(w_1, w_2) \approx \cos(f(w_1), f(w_2))$$

Learning Word Embeddings: Overview

The Core Idea

Train a neural network to predict words from their context. The **hidden layer weights** become our word embeddings.

Why this works:

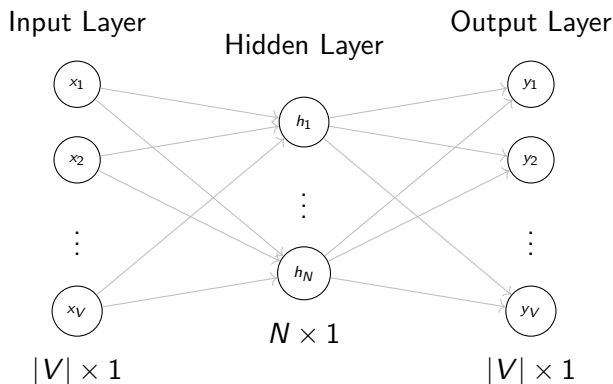
- 1 The network learns to predict which words appear together
- 2 Words that appear in similar contexts get similar representations
- 3 The hidden layer compresses one-hot vectors into dense embeddings

Key Insight: After training, the **input-to-hidden weights** are the word embeddings!

Three training strategies we'll cover:

- 1 Next word prediction (CBOW-like)
- 2 Center word prediction (Continuous Bag-of-Words)
- 3 Context-target prediction (Skip-gram)

The Neural Network Architecture



Layer	Size	Description
Input	$ V \times 1$	One-hot encoded word(s)
Hidden	$N \times 1$	Embedding vector ($N = 50-300$)
Output	$ V \times 1$	Probability distribution over vocabulary

Weight Matrices and Parameters

Two Weight Matrices

- W_1 (**Input** $\rightarrow$ **Hidden**): Size $N \times |V|$
- W_2 (**Hidden** $\rightarrow$ **Output**): Size $|V| \times N$

Input to Hidden:

$$W_1 = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1|V|} \\ w_{21} & w_{22} & \cdots & w_{2|V|} \\ \vdots & \vdots & \ddots & \vdots \\ w_{N1} & w_{N2} & \cdots & w_{N|V|} \end{bmatrix}$$

Hidden to Output:

$$W_2 = \begin{bmatrix} w'_{11} & w'_{12} & \cdots & w'_{1N} \\ w'_{21} & w'_{22} & \cdots & w'_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ w'_{|V|1} & w'_{|V|2} & \cdots & w'_{|V|N} \end{bmatrix}$$

Biases: b_1 (size $N \times 1$)

Biases: b_2 (size $|V| \times 1$)

Important

After training, **each column of W_1** is the embedding vector for a word!

Training Data Preparation: Method 1

Next Word Prediction (Autoregressive)

Predict the next word given the previous k words.

Example sentence: "The king sat on the throne"

Context ($k = 3$)	Target	Training Pair
[The, king, sat]	on	([The, king, sat], on)
[king, sat, on]	the	([king, sat, on], the)
[sat, on, the]	throne	([sat, on, the], throne)

Representing the Context (multiple words):

- Convert each word to one-hot: $x^{(1)}, x^{(2)}, \dots, x^{(k)}$
- Sum them: $x_{\text{context}} = \sum_{i=1}^k x^{(i)}$
- Or average: $x_{\text{context}} = \frac{1}{k} \sum_{i=1}^k x^{(i)}$

This is the Continuous Bag-of-Words (CBOW) approach

Training Data Preparation: Method 2

Center Word Prediction (CBOW)

Predict the center word given the surrounding context.

Example sentence: "The king sat on the throne" (window size = 2)

Context ($k = 4$)	Target	Training Pair
[The, king, on, the]	sat	(context, target)
[king, sat, the, throne]	on	(context, target)

Context representation: Sum or average of one-hot vectors

Training Data Preparation: Method 3 (Skip-gram)

Skip-gram

Predict surrounding context words given a target word.

Example sentence: "The king sat on the throne" (window size = 2)

Target	Context (within 2 words)	Training Pairs
sat	The, king, on	(The, sat), (king, sat), (on, sat)
king	The, sat, on	(The, king), (sat, king), (on, king)
throne	on, the	(on, throne), (the, throne)

Note: For Skip-gram, input is a **single word** (not averaged), output predicts each context word independently.

Forward Pass: Step by Step

Step 1: Input Layer Convert input word(s) to one-hot vector x (size $|V| \times 1$)

$$x = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad (1 \text{ at the index of the input word})$$

Step 2: Hidden Layer

$$h = \text{ReLU}(W_1 \cdot x + b_1)$$

Step 3: Output Layer (Logits)

$$z = W_2 \cdot h + b_2$$

Forward Pass: Softmax and Prediction

Step 4: Softmax Activation

Convert logits z to probabilities $\hat{y}$:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}} \quad \text{for } i = 1, 2, \dots, |V|$$

Example output:

$$\hat{y} = \begin{bmatrix} 0.02 \\ 0.01 \\ 0.85 \\ 0.03 \\ \vdots \\ 0.01 \end{bmatrix}$$

Word with highest probability (index 2) is the network's prediction.

The network learns to assign high probability to the correct target word.

Loss Function: Cross Entropy

True target y (one-hot):

$$y = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad (1 \text{ at the index of the correct target word})$$

Cross Entropy Loss:

$$\mathcal{L} = - \sum_{i=1}^{|V|} y_i \log(\hat{y}_i)$$

Since y is one-hot (only one $y_c = 1$):

$$\mathcal{L} = -\log(\hat{y}_c)$$

where c is the index of the correct target word.

Numerical Example: Loss Calculation

Setup

Vocabulary size $|V| = 5$, target word is "throne" (index 3)

True target y :

$$y = [0, 0, 0, 1, 0]^T$$

Network output $\hat{y}$ (after softmax):

$$\hat{y} = [0.05, 0.10, 0.15, 0.60, 0.10]^T$$

Compute loss:

$$\mathcal{L} = - \sum_{i=1}^5 y_i \log(\hat{y}_i) = 0.5108$$

What if prediction was poor?

$$\hat{y}_{poor} = [0.01, 0.01, 0.01, 0.01, 0.96] \rightarrow \mathcal{L} = -\log(0.01) = 4.6052$$

Higher loss = worse prediction!

Backpropagation

Goal

Update weights W_1, W_2, b_1, b_2 to minimize the loss $\mathcal{L}$.

Gradient Descent Update Rule:

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}$$

where η is the learning rate.

Chain Rule Application:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial W_2}$$

We need gradients for:

- ① $\frac{\partial \mathcal{L}}{\partial z}$ (output layer logits)
- ② $\frac{\partial \mathcal{L}}{\partial W_2}$ and $\frac{\partial \mathcal{L}}{\partial b_2}$
- ③ $\frac{\partial \mathcal{L}}{\partial h}$ (hidden layer)
- ④ $\frac{\partial \mathcal{L}}{\partial W_1}$ and $\frac{\partial \mathcal{L}}{\partial b_1}$

Complete Training Algorithm

1 Prepare training data:

- Choose method (next word, CBOW, or Skip-gram)
- Create (input, target) pairs from corpus
- Convert words to one-hot vectors

2 Initialize parameters:

- W_1 (size $N \times |V|$) with small random values
- W_2 (size $|V| \times N$) with small random values
- b_1 (size $N \times 1$) with zeros
- b_2 (size $|V| \times 1$) with zeros

3 For each epoch (multiple passes over data):

- For each training pair (x, y) :
 - Forward pass: compute $\hat{y}$
 - Compute loss: $\mathcal{L} = -\log(\hat{y}_c)$
 - Backward pass: compute gradients
 - Update weights using gradient descent

4 Extract embeddings: $embedding[i] = W_1[:, i]$

The Softmax Bottleneck Problem

Recall: Standard Word Embedding Training

For each training example, we compute:

$$\hat{y} = \text{softmax}(z) \quad \text{where} \quad \hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}}$$

The Problem: The denominator requires summing over **ALL** $|V|$ words!

Vocabulary Size	Operations per step	Time (approx.)
10,000	10,000	Fast
100,000	100,000	Slow
1,000,000	1,000,000	Very Slow
10,000,000	10,000,000	Impractical

The Issue

With $|V| = 1$ million, each training step requires **1 million exponential calculations**! This is too slow for practical training.

The Intuition Behind Negative Sampling

Instead of asking:

"Which word out of 1 million is the correct context?"

We ask: "Is this specific word a valid context for the target word?"

Key Insight: We only need to distinguish the real context from a few **random negatives**!

The Restaurant Analogy

Full Softmax (Waiter):

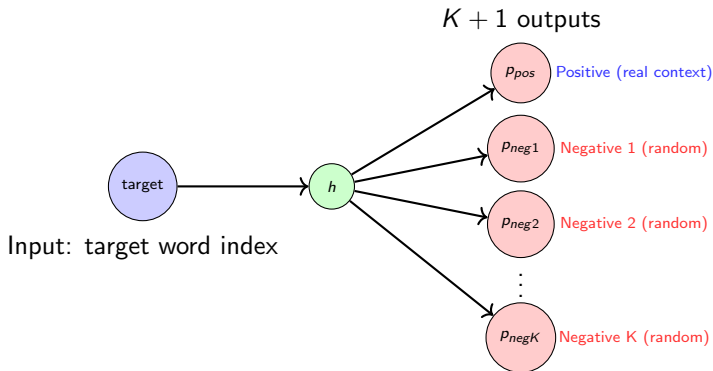
- Customer orders "something spicy"
- Waiter computes probability for **all 100 dishes**
- "Is it dish 1? Dish 2? ... Dish 100?"
- **Very slow!**

Negative Sampling (Smart Waiter):

- Customer orders "something spicy"
- Waiter picks:
 - 1 real dish the customer ordered
 - 4 random dishes from menu
- Only compares these 5 dishes!
- **Much faster!**

The model learns to distinguish real contexts from random noise.

The Negative Sampling Architecture



Key Difference from Full Softmax

- Instead of $|V|$ outputs, we have only $K + 1$ outputs ($K = 5$ to 20)
- Each output is a **binary classifier** (real context vs. fake)
- Massive speedup: $O((K + 1) \times D)$ instead of $O(|V| \times D)$

Network Architecture Details

Parameters (Same as Before!)

- W_1 : Input embedding matrix ($N \times |V|$) — each column = word embedding
- W_2 : Output embedding matrix ($|V| \times N$) — context vectors
- We still store ALL $|V|$ vectors in memory
- But we only **use/update** $K + 1$ of them per step!

Per Training Step:

Component	Operation
Input lookup	$h = W_1[:, \text{target_idx}]$
Positive branch	$\text{score}_{\text{pos}} = h \cdot W_2[\text{pos_idx}]$
Negative branches	$\text{score}_{\text{neg}_j} = h \cdot W_2[\text{neg_idx}_j]$
Output	$\sigma(\text{score})$ for each branch

Only $K + 1$ rows of W' are accessed per step — the rest are untouched!

The Loss Function

Binary Cross-Entropy Loss

For each training example with 1 positive and K negative samples:

$$\mathcal{L} = -\log(p_{pos}) - \sum_{j=1}^K \log(1 - p_{neg_j})$$

Intuition:

Branch	Desired p	Loss contribution
Positive	≈ 1	$-\log(1) = 0$
Negative	≈ 0	$-\log(1 - 0) = -\log(1) = 0$

If predictions are bad:

$$p_{pos} = 0.1 \Rightarrow -\log(0.1) = 2.30 \quad (\text{high loss})$$

$$p_{neg} = 0.9 \Rightarrow -\log(0.1) = 2.30 \quad (\text{high loss})$$

How Negative Words Are Sampled

Not Uniform Random!

We sample negatives based on word frequency raised to $3/4$ power:

$$P(w) \propto f(w)^{3/4}$$

Why $3/4$ power?

Word	Frequency (f)	$f^{3/4}$	Relative Probability
"the"	1,000,000	1,000,000	6.7%
"king"	10,000	1,000	0.007%
"pizza"	500	105	0.0007%

Effect:

- Common words (like "the") are sampled **less often** than raw frequency
- Rare words (like "pizza") are sampled **more often** than raw frequency
- Prevents common words from dominating negative samples

Choosing K (Number of Negative Samples)

Rule of Thumb

- Small datasets: $K = 5$ to 10
- Large datasets: $K = 10$ to 20
- Very large datasets: $K = 20$ to 30

Trade-offs:

K	Speed	Quality
1	Very Fast	Poor (not enough negatives)
5-10	Fast	Good
15-20	Moderate	Better
50+	Slow	Diminishing returns

Key Insight

More negatives = better discrimination, but slower training.

Word2Vec (Predictive Model)

- Uses a **sliding window** over the corpus
- Creates (target, context) pairs from local context
- Trains a neural network to predict context from target (Skip-gram) or vice versa (CBOW)
- Uses **negative sampling** for efficiency

Strengths:

- Fast training on very large corpora
- Captures local syntactic patterns
- Simple architecture

Limitations:

- Only uses **local** context (window of k words)
- Ignores **global** corpus statistics
- Doesn't leverage word co-occurrence counts efficiently

Introducing GloVe (Global Vectors)

GloVe: Global Vectors for Word Representation

Proposed by Pennington, Socher, and Manning (2014)

Core Idea: Combine the best of both worlds:

- **Word2Vec's** local context window (predictive)
- **Matrix factorization's** global statistics (count-based)

Key Insight: Use the entire corpus's **co-occurrence statistics** to learn embeddings, not just local windows.

Word2Vec vs GloVe:

Aspect	Word2Vec	GloVe
Context	Local (window)	Global (full corpus)
Training	Online (pairs)	Count matrix + optimization
Speed	Fast on large data	Needs co-occurrence matrix

The Co-occurrence Matrix

Definition

X_{ij} = number of times word j appears in the context of word i (within a window of size k)

Example Corpus: "the cat sat on the mat"

	the	cat	sat	on	mat
the	0	2	1	1	1
cat	2	0	1	0	0
sat	1	1	0	1	0
on	1	0	1	0	1
mat	1	0	0	1	0

Properties:

- Size: $|V| \times |V|$ (can be very large!)
- Sparse: most entries are zero
- Captures **global** corpus statistics

The Core Idea: What Should Embeddings Capture?

The Goal

Learn embeddings such that the **dot product** between two word vectors reflects their **co-occurrence** in the corpus.

Intuition:

- If words i and j frequently appear together, their dot product should be LARGE
- If words i and j rarely appear together, their dot product should be SMALL

Proposed relationship:

$$w_i \cdot \tilde{w}_j \propto \text{something related to } X_{ij}$$

where X_{ij} = number of times j appears in context of i

The Problem with Raw Co-occurrence Counts

Why can't we just set $w_i \cdot \tilde{w}_j = X_{ij}$?

Co-occurrence counts follow a **power law distribution** (Zipf's law).

Example: Typical co-occurrence counts

Word Pair	X_{ij}
(the, of)	1,000,000
(king, throne)	5,000
(king, queen)	500
(king, pizza)	1

The Problem:

- Range is **enormous** (1 to 1,000,000+)
- Common pairs dominate the loss
- Differences between rare pairs get lost
- Embeddings would be dominated by frequent words

The Power Law and Log Transformation

Solution: Take the logarithm!

X_{ij}	$\log X_{ij}$	Effect
1,000,000	≈ 13.8	Large, but compressed
5,000	≈ 8.5	Medium
500	≈ 6.2	Medium-small
1	0	Smallest

Log transform compresses the scale and makes relationships more linear!

Key Takeaway

The dot product of word embeddings of word i and j can be approximated to make it equal to $\log X_{ij}$

The GloVe Objective Function

The GloVe Loss Function

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \cdot (w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

Where:

- w_i : embedding vector for word i (target)
- $\tilde{w}_j$: embedding vector for word j (context)
- $b_i, \tilde{b}_j$: bias terms
- X_{ij} : co-occurrence count
- $f(X_{ij})$: weighting function

What this means:

- We want $w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j$ to be close to $\log X_{ij}$
- Weighted least squares regression!
- Different from Word2Vec's binary cross-entropy

The Weighting Function $f(X_{ij})$

Why do we need weighting?

Not all co-occurrences are equally important!

The weighting function:

$$f(x) = \begin{cases} (x/x_{max})^\alpha & \text{if } x < x_{max} \\ 1 & \text{otherwise} \end{cases}$$

Typical values:

- $x_{max} = 100$ (cap for very frequent pairs)
- $\alpha = 3/4$ (0.75)

X_{ij}	$f(X_{ij})$	Why?
0	0	Skip (no co-occurrence)
10	0.18	Low weight
50	0.59	Medium weight
1000	1.00	Capped

Extracting Final Embeddings

GloVe learns two sets of embeddings

- w_i : target word embeddings
- $\tilde{w}_j$: context word embeddings

Which one to use? Both are good! Usually we take the **sum** or **average**:

$$\text{final_embedding} = w_i + \tilde{w}_j$$

or

$$\text{final_embedding} = \frac{w_i + \tilde{w}_j}{2}$$

Why both work?

- The model is symmetric in i and j
- Both capture meaningful relationships
- Averaging reduces noise

GloVe Training Algorithm Summary

① Build co-occurrence matrix:

- Scan corpus once, count X_{ij} for all word pairs within window
- Store non-zero entries in sparse format

② Initialize parameters:

- w_i and $\tilde{w}_i$ for all words (random small values)
- b_i and $\tilde{b}_i$ (zeros)

③ For each epoch (multiple passes over non-zero pairs):

- Shuffle training pairs (important for convergence)
- For each (i, j, X_{ij}) :
 - Compute prediction $= w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j$
 - Compute error $\epsilon = \text{prediction} - \log X_{ij}$
 - Compute weight $f(X_{ij})$
 - Update $w_i, \tilde{w}_j, b_i, \tilde{b}_j$ using gradients

④ Extract final embeddings: $\text{embedding}_i = \frac{w_i + \tilde{w}_i}{2}$

GloVe vs Word2Vec - Architecture Comparison

Word2Vec (Skip-gram):

- Neural network
- Predicts context from target
- Binary cross-entropy loss
- Negative sampling
- Local windows only

GloVe:

- Matrix factorization
- Predicts $\log X_{ij}$ from $w_i \cdot \tilde{w}_j$
- Weighted MSE loss
- No negative sampling
- Global co-occurrence matrix

	Word2Vec	GloVe
Training data	(target, context) pairs	Co-occurrence matrix
Loss function	Binary CE	Weighted MSE
Output activation	Sigmoid	Linear
Sampling needed	Yes (negatives)	No
Captures	Local patterns	Global statistics